1. Enteder el problema

El análisis del Titanic, particularmente en el contexto de machine learning y ciencia de datos, generalmente se refiere al uso del conjunto de datos de pasajeros del Titanic para resolver problemas de predicción. El problema central es predecir quiénes sobrevivieron al hundimiento basándose en ciertas características como la clase del boleto, el sexo, la edad, entre otros.

```
# Importar las librerias
import pandas as pd
```

2. La ingesta de datos en ciencia de datos

Se refiere al proceso de recopilar, importar y cargar datos desde diversas fuentes para su posterior análisis, almacenamiento o procesamiento en sistemas de data analytics o machine learning. Es el primer paso crucial dentro de un flujo de trabajo de ciencia de datos, ya que garantiza que los datos necesarios estén disponibles en el sistema y sean accesibles para ser manipulados y analizados.

El proceso de ingesta de datos puede incluir varias etapas:

Extracción de datos: Los datos se extraen de diferentes fuentes, como bases de datos relacionales, archivos CSV, APIs, dispositivos IoT, flujos de datos en tiempo real o datos en la nube.

Transformación de datos: Antes de cargar los datos en el sistema final, a menudo es necesario aplicar transformaciones como la limpieza, normalización, estandarización o formateo de los datos para adaptarlos a los requisitos del análisis posterior.

Carga de datos: Los datos se cargan en sistemas de almacenamiento de datos como bases de datos, lagos de datos (data lakes) o almacenes de datos (data warehouses), donde estarán disponibles para su análisis.

```
# Cargar el archivo CSV del Titanic
df = pd.read_csv('titanic.csv')

# Mostrar las primeras filas del dataset
df.head()
```

	PassengerId	Survived	Pclass	Lname	Name	Sex	Age	SibSp	Parch	Tick
0	1	0	3	Braund	Mr. Owen Harris	male	22.0	1	0	A/5 211
1	2	1	1	Cumings	Mrs. John Bradley (Florence Briggs	female	38.0	1	0	PC 175

3. Analisis exploratorio de datos (EDA en inglés)

Descripción: En esta etapa, se examinan los datos para entender sus características principales. Esto incluye identificar valores faltantes, realizar visualizaciones, buscar patrones y detectar posibles anomalías. Técnicas comunes: Estadísticas descriptivas, visualizaciones con gráficos de dispersión, histogramas, diagramas de correlación, etc. Ejemplo: Determinar si hay correlación entre la antigüedad de un cliente y su probabilidad de abandono.

```
#EDA
# Información general del dataset
df.info()

# Resumen estadístico de las variables numéricas
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 156 entries, 0 to 155
Data columns (total 13 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  156 non-null    int64
 1   Survived     156 non-null    int64
 2   Pclass       156 non-null    int64
 3   Lname        156 non-null    object
 4   Name         156 non-null    object
 5   Sex          156 non-null    object
 6   Age          126 non-null    float64
 7   SibSp        156 non-null    int64
 8   Parch        156 non-null    int64
 9   Ticket       156 non-null    object
 10  Fare         156 non-null    float64
 11  Cabin        31 non-null     object
 12  Embarked     155 non-null    object
dtypes: float64(2), int64(5), object(6)
memory usage: 16.0+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Far
count	156.000000	156.000000	156.000000	126.000000	156.000000	156.000000	156.00000
mean	78.500000	0.346154	2.423077	28.141508	0.615385	0.397436	28.10958
std	45.177428	0.477275	0.795459	14.613880	1.056235	0.870146	39.40104
min	1.000000	0.000000	1.000000	0.830000	0.000000	0.000000	6.75000
25%	39.750000	0.000000	2.000000	19.000000	0.000000	0.000000	8.00315
50%	78.500000	0.000000	3.000000	26.000000	0.000000	0.000000	14.45420
75%	117.250000	1.000000	3.000000	35.000000	1.000000	0.000000	30.37185
max	156.000000	1.000000	3.000000	71.000000	5.000000	5.000000	263.00000

```
# Verificar valores nulos en cada columna
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Lname	0
Name	0
Sex	0
Age	30
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	125
Embarked	1

dtype: int64

3. Preprocesamiento y Limpieza de Datos

Descripción: Eliminar o corregir datos incorrectos, duplicados, inconsistentes o irrelevantes. También puede implicar la imputación de valores faltantes y el tratamiento de datos atípicos (outliers).

Ejemplo: Rellenar valores faltantes en las columnas de edad o ingresos de clientes, o eliminar registros duplicados.

```
# Imputar la edad con la media
df['Age'].fillna(df['Age'].mean(), inplace=True)

# Rellenar los valores faltantes de la columna 'Embarked' con el valor más frecuente
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
# Para la columna 'Cabin', una opción es eliminarla ya que muchos valores están faltar
df.drop(columns=['Cabin'], inplace=True)
```

```
<ipython-input-5-4cae0456690d>:2: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({cc


  df['Age'].fillna(df['Age'].mean(), inplace=True)
<ipython-input-5-4cae0456690d>:5: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({cc


  df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
# Verificar valores nulos en cada columna
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Lname	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Embarked	0

dtype: int64

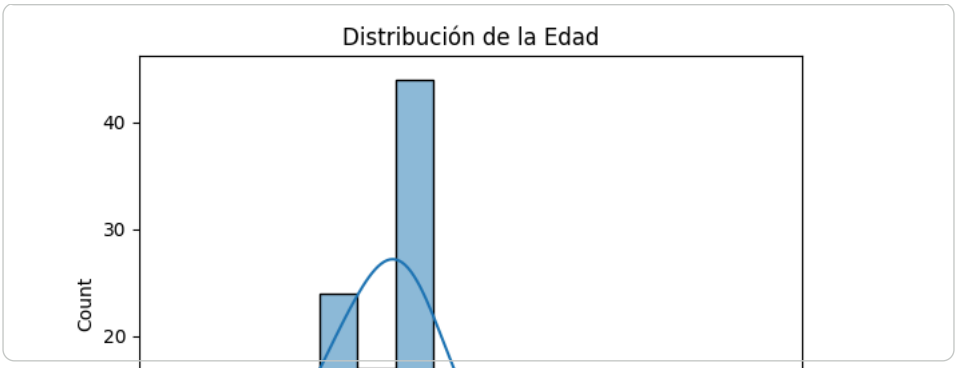
```
#EDA
import seaborn as sns
import matplotlib.pyplot as plt

# Gráfico de distribución de la edad
sns.histplot(df['Age'], kde=True)
plt.title('Distribución de la Edad')
plt.show()

# Gráfico de conteo de supervivientes
sns.countplot(x='Survived', data=df)
plt.title('Supervivientes')
plt.show()

# Gráfico de supervivencia según el sexo
sns.countplot(x='Survived', hue='Sex', data=df)
plt.title('Supervivencia según el Sexo')
plt.show()

# Gráfico de supervivencia según la clase del pasajero
sns.countplot(x='Survived', hue='Pclass', data=df)
plt.title('Supervivencia según la Clase del Pasajero')
plt.show()
```



4. Modelado

Selección de modelo Descripción: Elegir el algoritmo de machine learning o modelo analítico adecuado para resolver el problema. La selección depende de la naturaleza del problema: clasificación, regresión, agrupamiento, etc. Ejemplo: Elegir entre un modelo de regresión logística, árboles de decisión o redes neuronales para predecir el abandono de clientes.

```
# Tecnicas de machine Learning

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Convertir las variables categóricas en variables dummy (0/1)
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)

# Seleccionar las variables predictoras y la variable objetivo
X = df[['Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Sex_male', 'Embarked_Q', 'Embarkec
y = df['Survived']

# Dividir los datos en conjunto de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=

# Crear el modelo de regresión logística
logmodel = LogisticRegression()
logmodel.fit(X_train, y_train)

# Predecir en los datos de prueba
y_pred = logmodel.predict(X_test)

# Evaluar el modelo
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

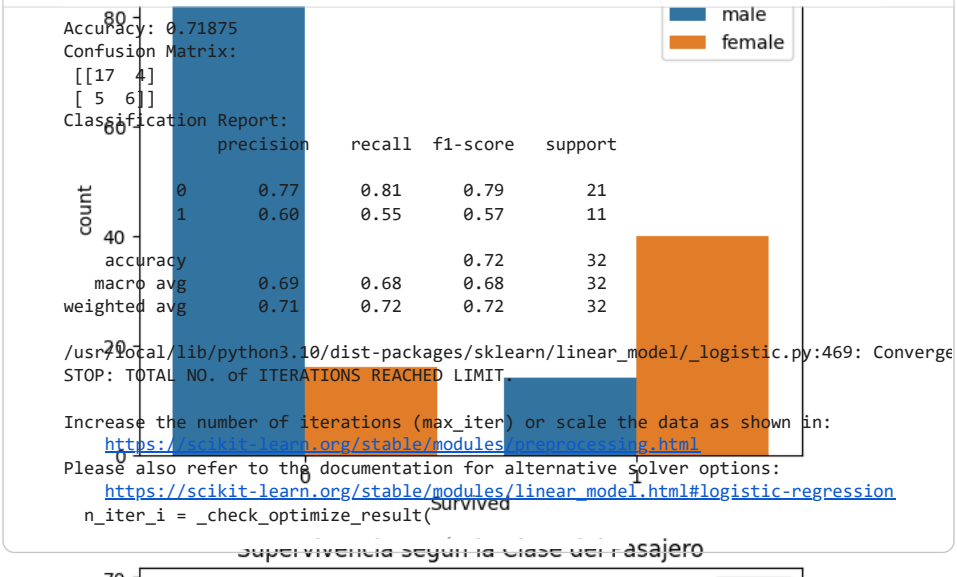
```
Accuracy: 0.71875
Confusion Matrix:
 [[17  4]
 [ 5  6]]
Classification Report:
              precision    recall  f1-score   support

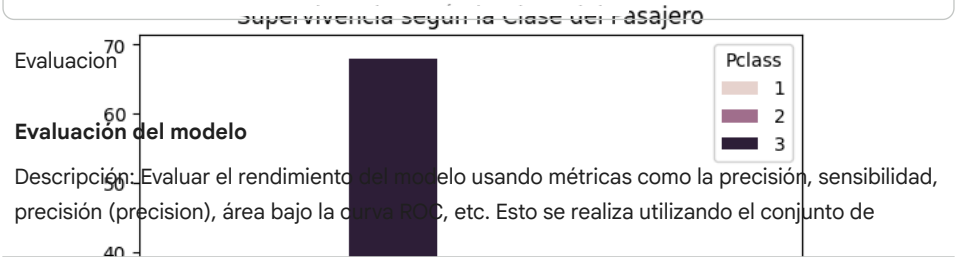
           0       0.77      0.81      0.79        21
           1       0.60      0.55      0.57        11

    accuracy                           0.72        32
   macro avg       0.69      0.68      0.68        32
weighted avg       0.71      0.72      0.72        32

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:469: Converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```





Evaluacion

Evaluación del modelo

Descripción: Evaluar el rendimiento del modelo usando métricas como la precisión, sensibilidad, precisión (precision), área bajo la curva ROC, etc. Esto se realiza utilizando el conjunto de

prueba que no ha sido visto por el modelo durante el entrenamiento

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, r

# Predicciones en los datos de prueba
y_pred = logmodel.predict(X_test)

# 1. Precisión del modelo
accuracy = accuracy_score(y_test, y_pred)
print(f"Precisión del modelo: {accuracy:.2f}")

# 2. Matriz de confusión
conf_matrix = confusion_matrix(y_test, y_pred)
print("Matriz de confusión:")
print(conf_matrix)

# 3. Informe de clasificación
class_report = classification_report(y_test, y_pred)
print("Informe de clasificación:")
print(class_report)

# 4. ROC y AUC (Opcional)
y_pred_prob = logmodel.predict_proba(X_test)[:,1]  # Predicciones de probabilidad para
roc_auc = roc_auc_score(y_test, y_pred_prob)
print(f"AUC-ROC: {roc_auc:.2f}")

# Gráfico ROC
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('Tasa de Falsos Positivos (FPR)')
plt.ylabel('Tasa de Verdaderos Positivos (TPR)')
plt.title('Curva ROC')
plt.legend()
plt.show()
```

```
Precisión del modelo: 0.72
Matriz de confusión:
[[17  4]
 [ 5  6]]
Informe de clasificación:
              precision    recall  f1-score   support

           0       0.77      0.81      0.79        21
           1       0.60      0.55      0.57        11

    accuracy                           0.72        32
   macro avg       0.69      0.68      0.68        32
weighted avg       0.71      0.72      0.72        32

AUC-ROC: 0.80
```

